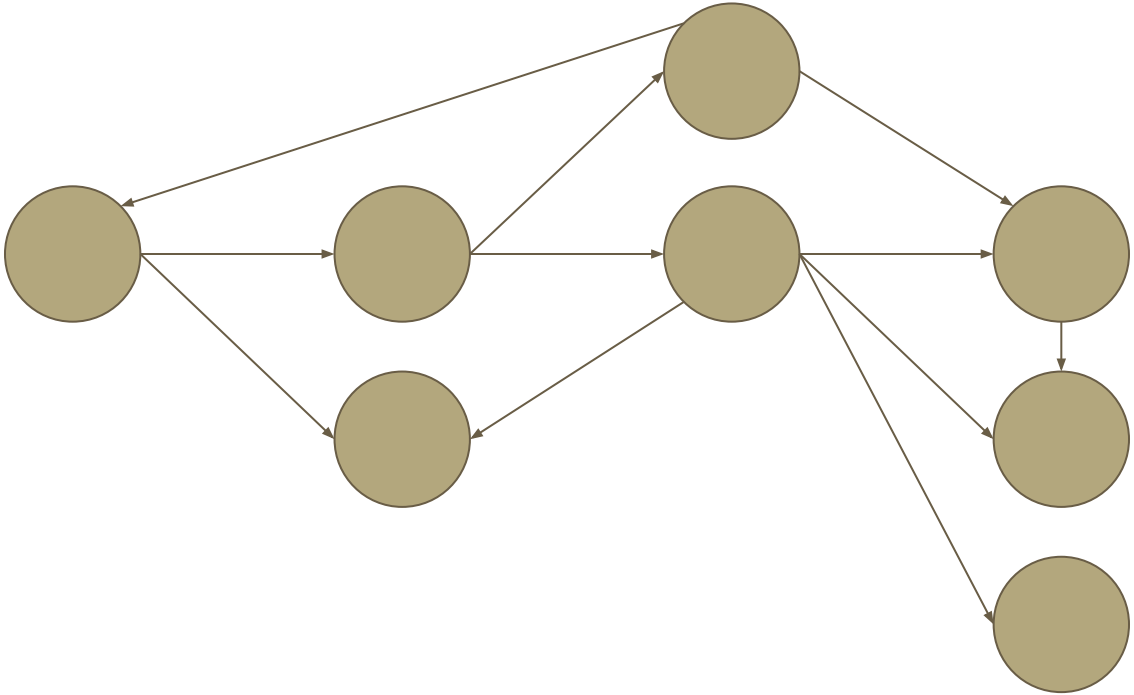
Estructuras de Datos

2026
Segundo Cuatrimestre

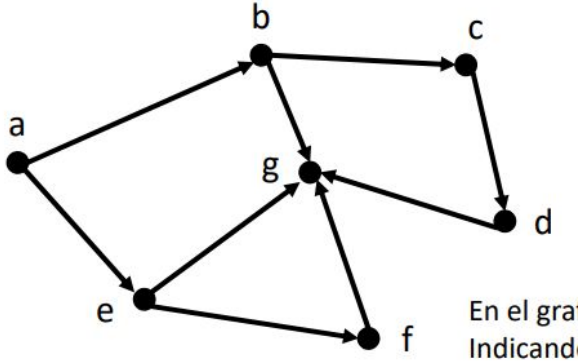
REPASO

Grafo



Grafo Dirigido/Digrafo

Un grafo G=(V,E) es un conjunto V de vértices (o nodos) y un conjunto E de aristas dirigidas (VxV) o arcos dirigidos. Intuitivamente E permite representar una relación entre elementos de V.

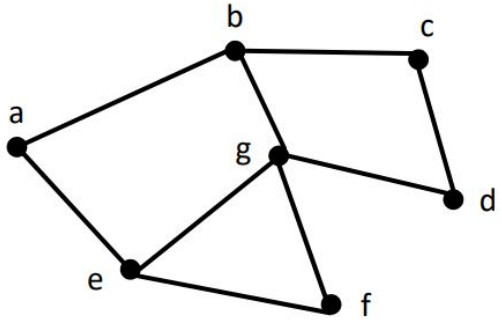


V = { a, b, c, d, e, f, g }
E = { (a,b), (b,c), (c,d), (d,g), (e,g), (e,f), (f,g), (a,e), (b,g) }

En el grafo dirigido, el arco (u,v) es un par ordenado
Indicando una flecha del vértice u al vértice v.

Grafo no Dirigido

Un grafo G=(V,E) es un conjunto V de vértices (o nodos) y un conjunto E de aristas (VxV) o arcos. Intuitivamente E permite representar una relación (simétrica) entre elementos de V.



V = { a, b, c, d, e, f, g }
E = { (a,b), (b,c), (c,d), (d,g), (e,g), (e,f), (f,g), (a,e), (b,g) }

En el grafo no dirigido, el arco (u,v) en realidad es un conjunto {u,v} ya que (u,v) es lo mismo que (v,u).

TDA Grafo no Dirigido

```
public interface Graph<V, E> {
        public Iterable<Vertex<V>> vertices();
        public Iterable<Edge<E>> edges();
        public Iterable<Edge<E>> incidentEdges(Vertex<V> v);
        public Vertex<V> opposite(Vertex<V> v, Edge<E> e);
        public Vertex<V> [] endVertices(Edge<E> e);
        public boolean areAdjacent(Vertex<V> v, Vertex<V> w);
        public V replace( Vertex<V> v, V x);
        public E replace( Edge<E> e, E x);
        public Vertex<V> insertVertex(V x);
        public Edge<E> insertEdge(Vertex<V> v, Vertex<V> w, E x);
        public V removeVertex(Vertex<V> v);
        public E removeEdge(Edge<E> e); }
```

IMPLEMENTANDO EL TDA GRAFO

Estructuras de Datos para Grafos

- Lista de Arcos
- Lista de Adyacencias
- Matriz de Adyacencia

Estructuras de Datos para Grafos

Lista de Arcos

- El grafo es una lista de vértices y una lista de arcos
- Los vértices
  - conocen sus rótulos respectivos
- Los arcos
  - conocen sus rótulos respectivos,
  - conocen los vértices que unen.

Estructuras de Datos para Grafos

Lista de Adyacencias

- El grafo es una lista de vértices y una lista de arcos
- Los vértices
  - conocen su rótulo,
  - conocen los arcos que inciden en él.
- Los arcos
  - conocen los vértices que unen,
  - conocen su rótulo.

Estructuras de Datos para Grafos

Matriz de Adyacencia

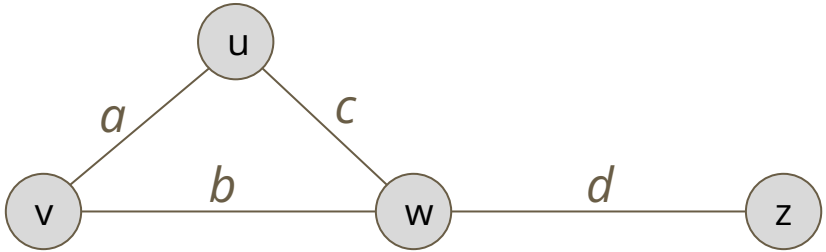
- El grafo es una lista de vértices, una lista de arcos y una matriz donde cada componente (i,j) almacena un arco.
- Los vértices
  - conocen su rótulo.
- Los arcos
  - conocen su rótulo.

Estructuras de Datos para Grafos

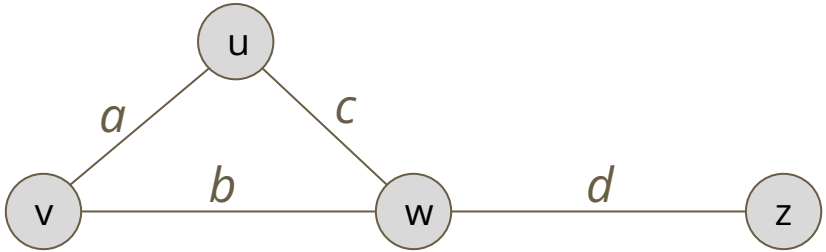
Lista de Arcos

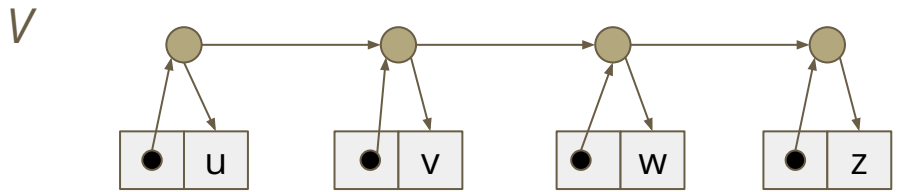
- El grafo es una lista de vértices y una lista de arcos
- Los vértices
  - conocen sus rótulos respectivos,
  - conocen su posición en la lista de vértices
- Los arcos
  - conocen sus rótulos respectivos,
  - conocen los vértices que unen,
  - conocen su posición en la lista de arcos

Estructuras para Grafos / Lista de Arcos

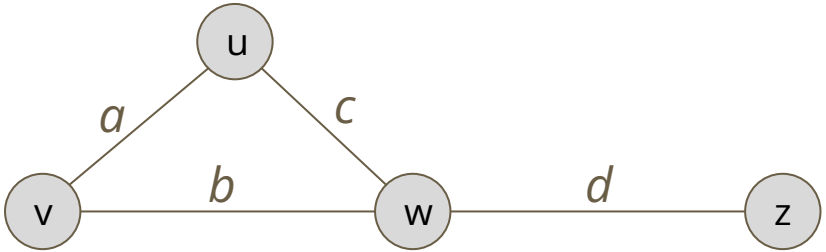


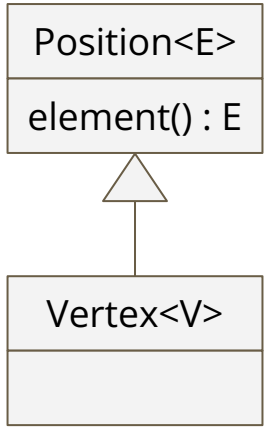
Estructuras para Grafos / Lista de Arcos



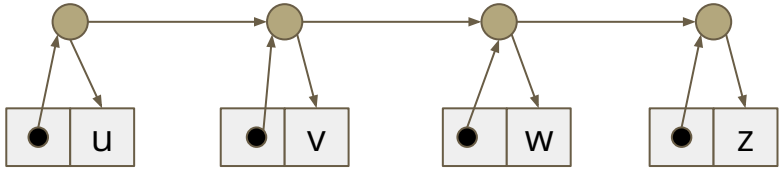


Estructuras para Grafos / Lista de Arcos

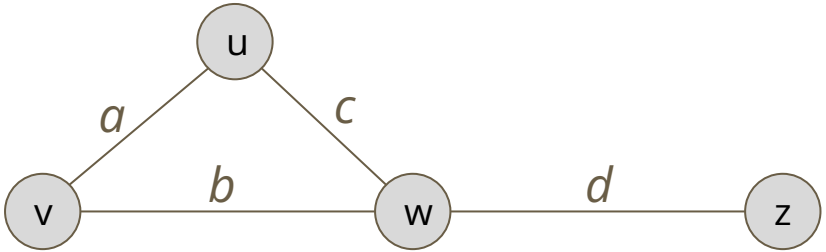




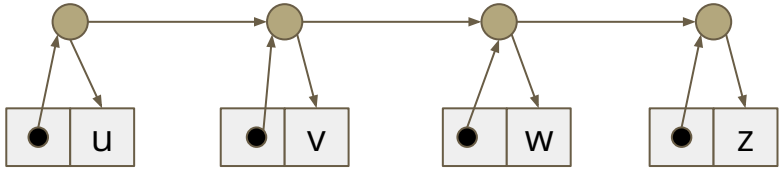
V

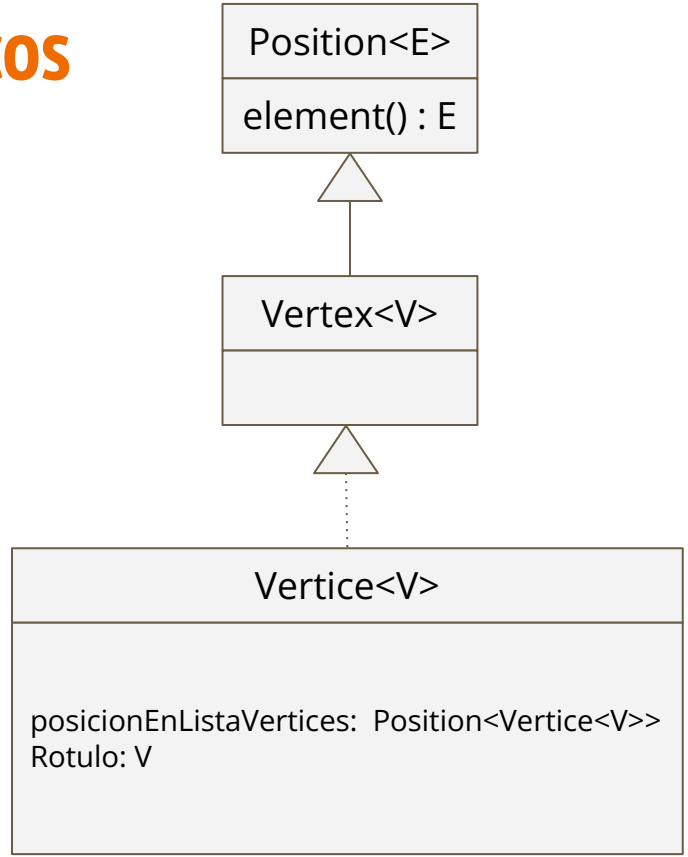


Estructuras para Grafos / Lista de Arcos

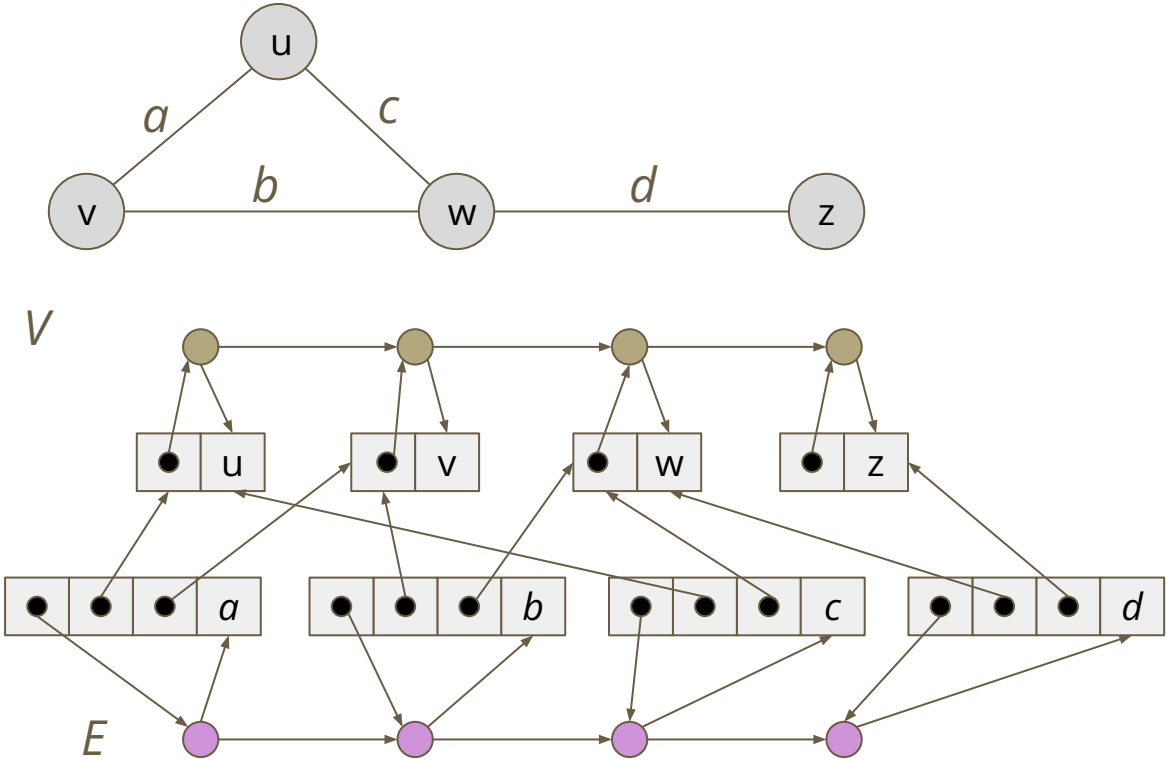


V

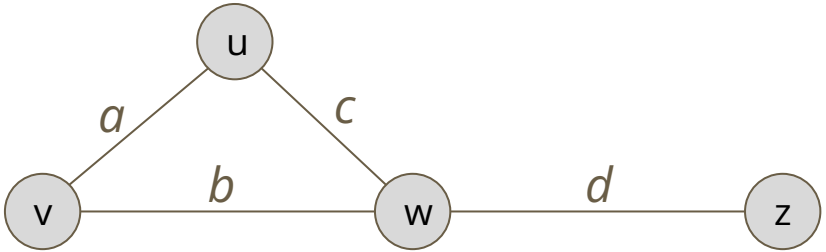


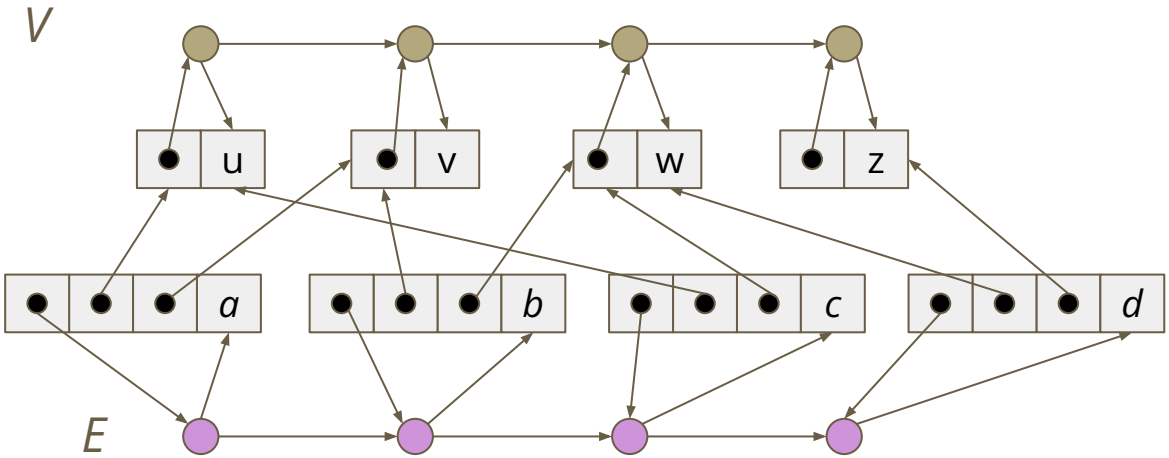


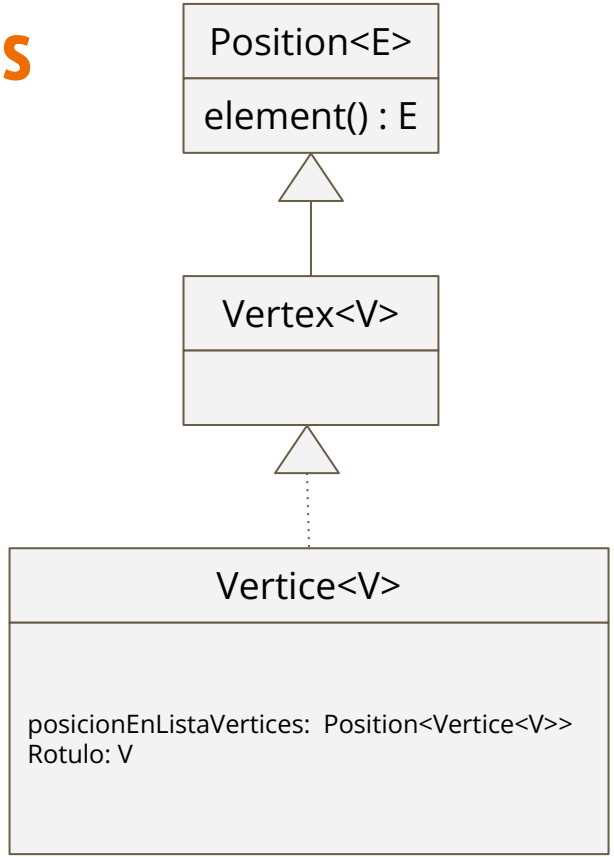
Estructuras para Grafos / Lista de Arcos



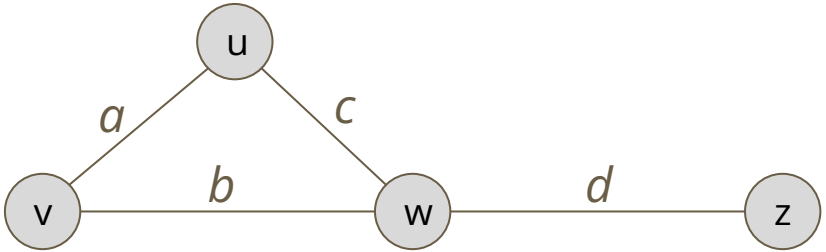
Estructuras para Grafos / Lista de Arcos

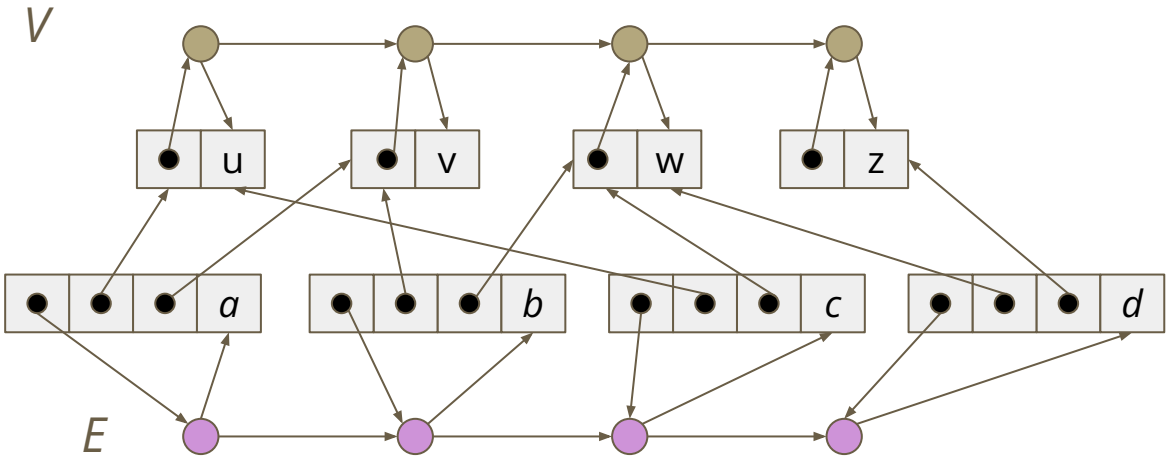


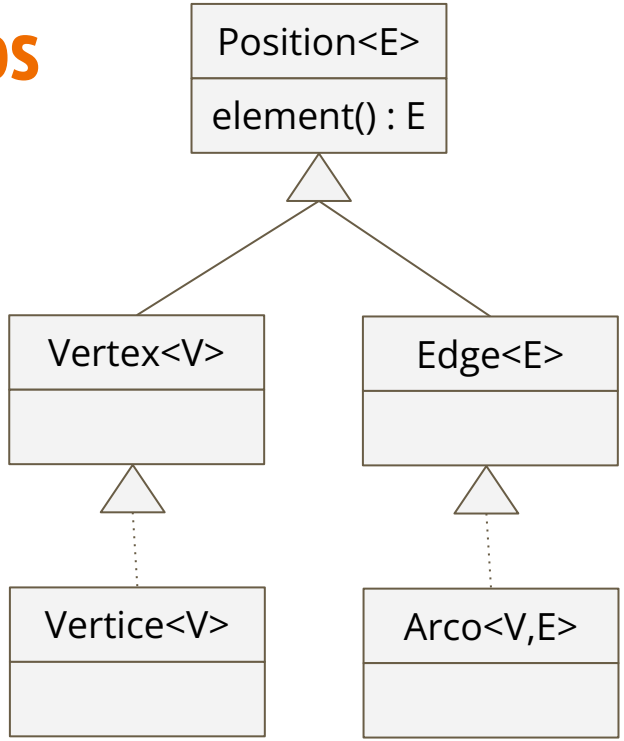




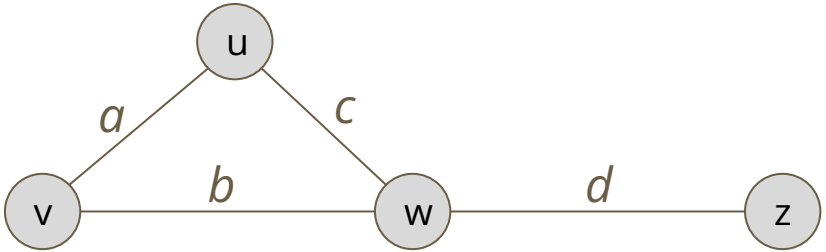
Estructuras para Grafos / Lista de Arcos

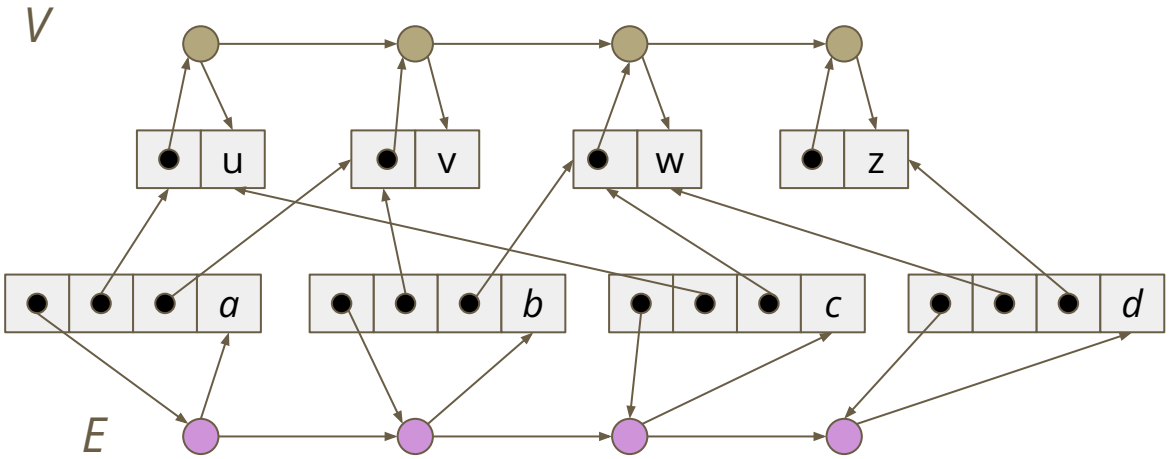


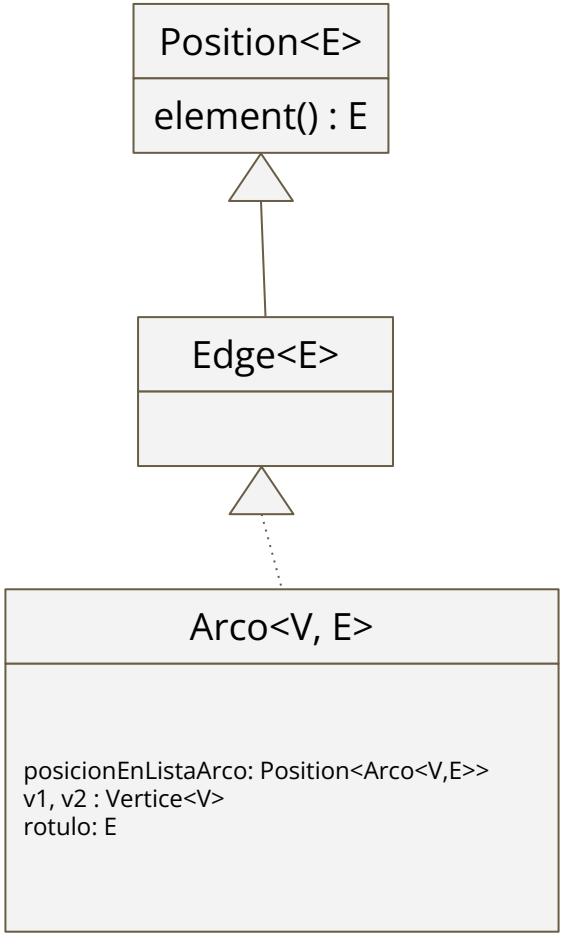




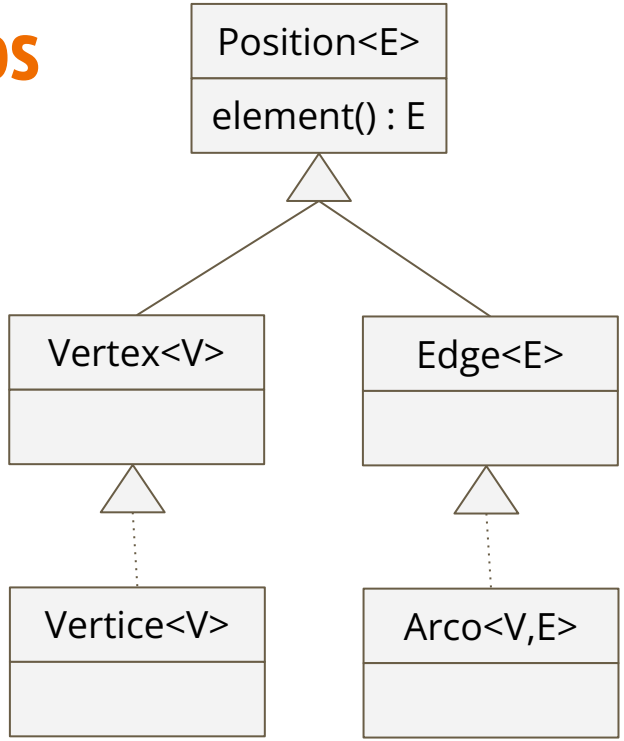
Estructuras para Grafos / Lista de Arcos



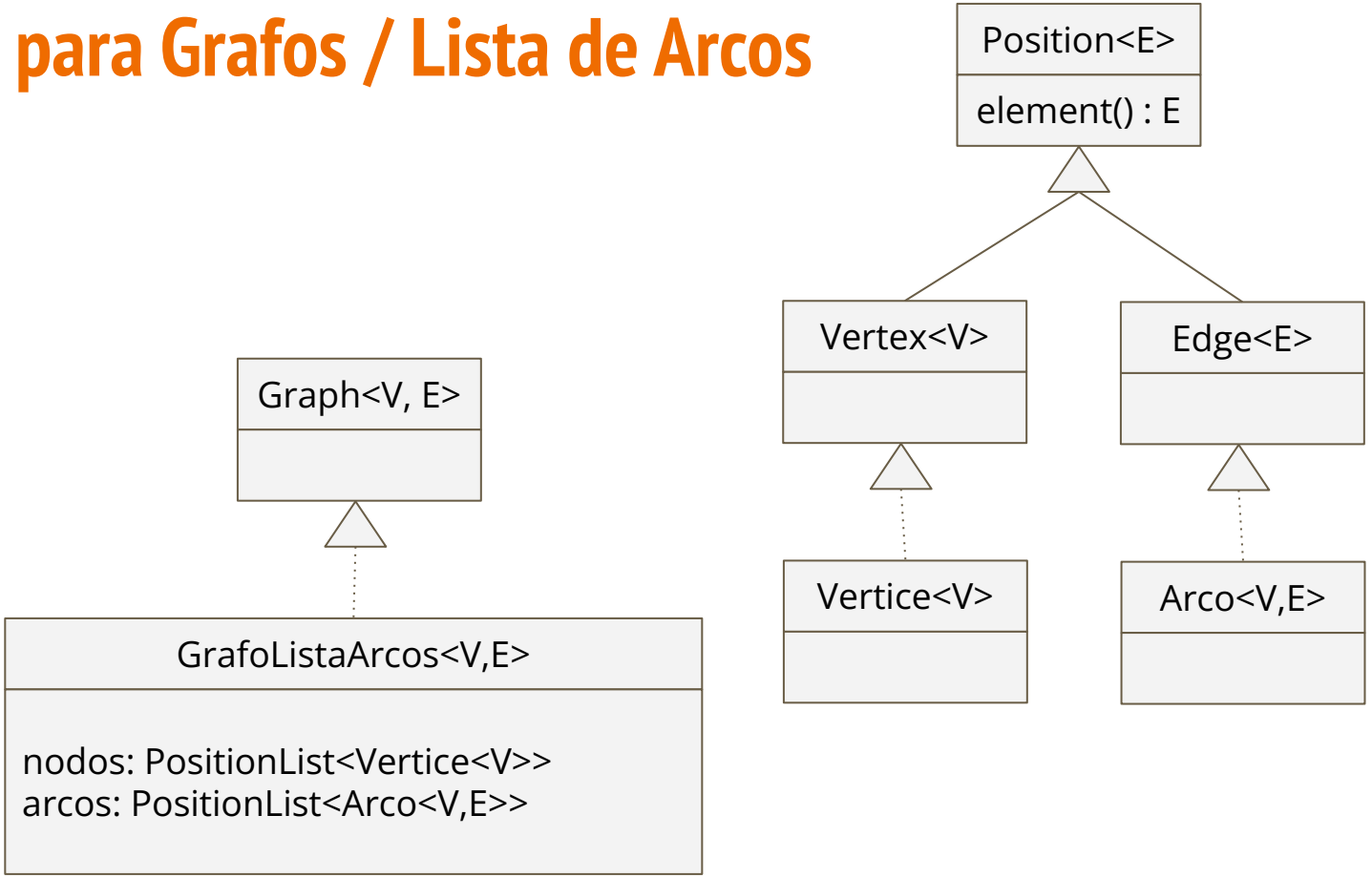




Estructuras para Grafos / Lista de Arcos



Estructuras para Grafos / Lista de Arcos



Estructuras para Grafos / Lista de Arcos

incidentEdges(Vertex<V> v);

opposite(Vertex<V> v, Edge<E> e);

endVertices(Edge<E> e);

areAdjacent(Vertex<V> v, Vertex<V> w);

replace(Vertex<V> v, V x);

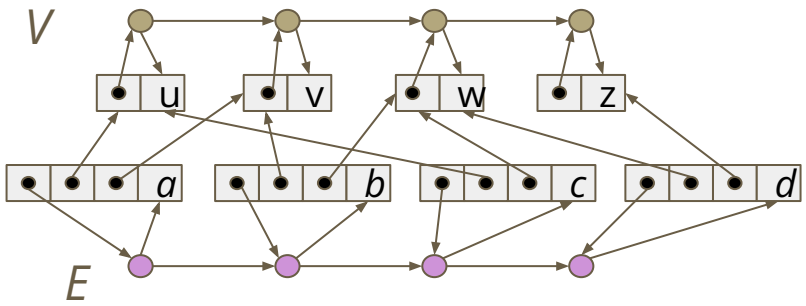
replace(Edge<E> e, E x);

insertVertex(V x);

insertEdge(Vertex<V> v, Vertex<V> w, E x);

removeVertex(Vertex<V> v);

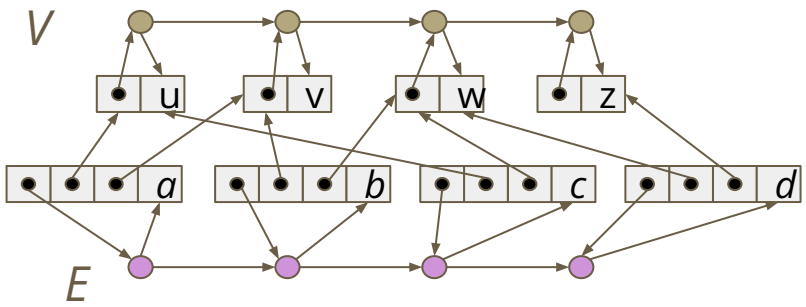
removeEdge(Edge<E> e); }



Dado un grafo G=(V,A). Sea n = #V y m = #A

Estructuras para Grafos / Lista de Arcos

incidentEdges(Vertex<V> v);	O(m)
opposite(Vertex<V> v, Edge<E> e);	O(1)
endVertices(Edge<E> e);	O(1)
areAdjacent(Vertex<V> v, Vertex<V> w);	O(m)
replace(Vertex<V> v, V x);	O(1)
replace(Edge<E> e, E x);	O(1)
insertVertex(V x);	O(1)
insertEdge(Vertex<V> v, Vertex<V> w, E x);	O(1)
removeVertex(Vertex<V> v);	O(m)
removeEdge(Edge<E> e); }	O(1)



Dado un grafo G=(V,A). Sea n = #V y m = #A

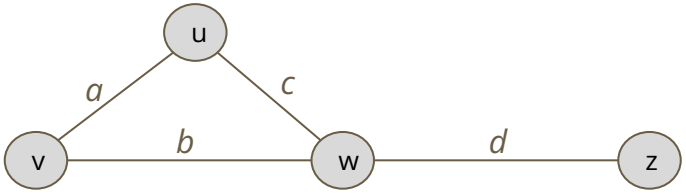
Estructuras de Datos para Grafos

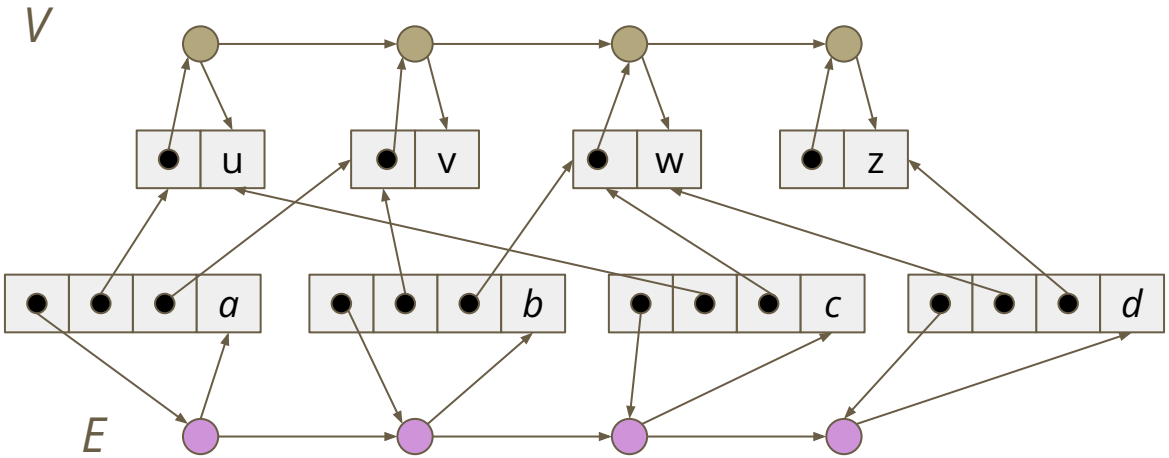
Lista de Adyacencias

- El grafo conoce una lista de vértices y una lista de arcos
- Los vértices
  - conocen su rótulo,
  - conocen los arcos que inciden en él.
- Los arcos
  - conocen los vértices que unen,
  - conocen su rótulo.

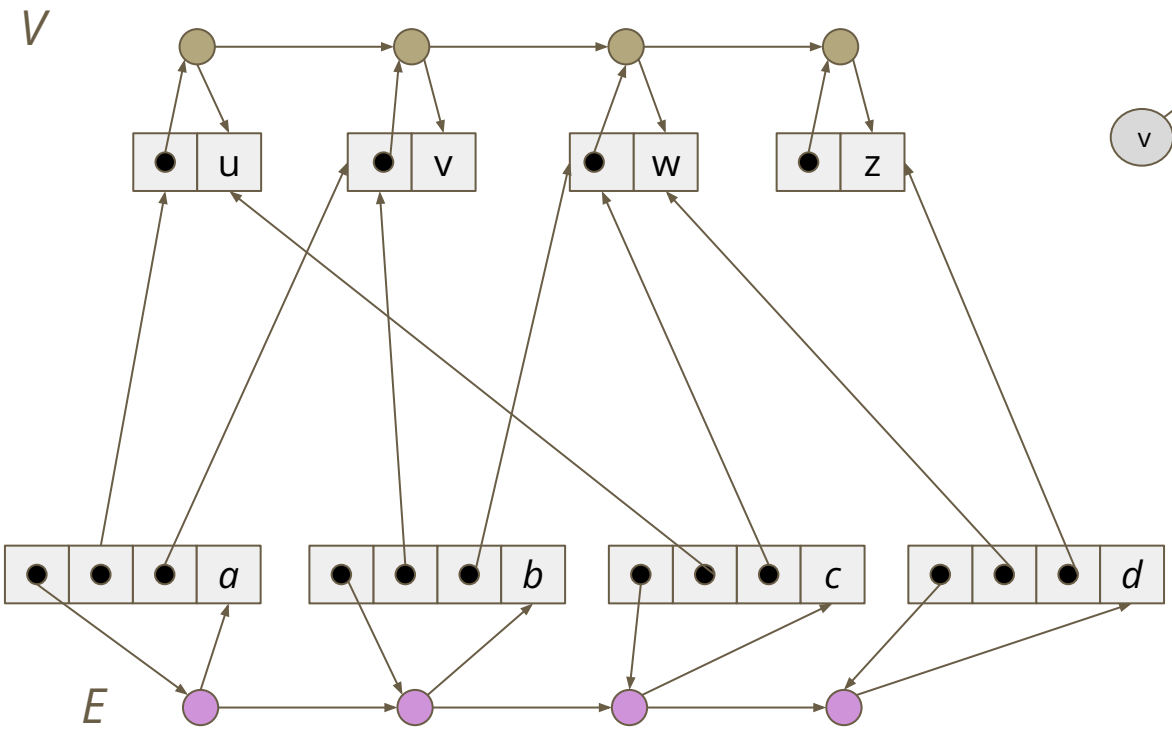
La mejora que propone la lista de adyacencias es introducir un lista que, para cada vértice, indica los arcos que emergen cuando el grafo es dirigido y los que emergen/inciden cuando el grafo es no dirigido.

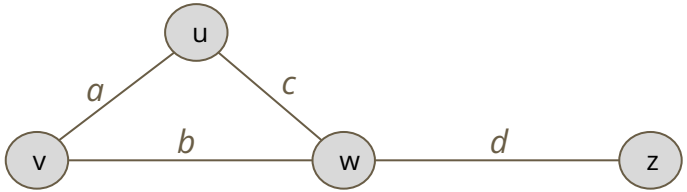
Estructuras para Grafos / Lista de Adyacencia



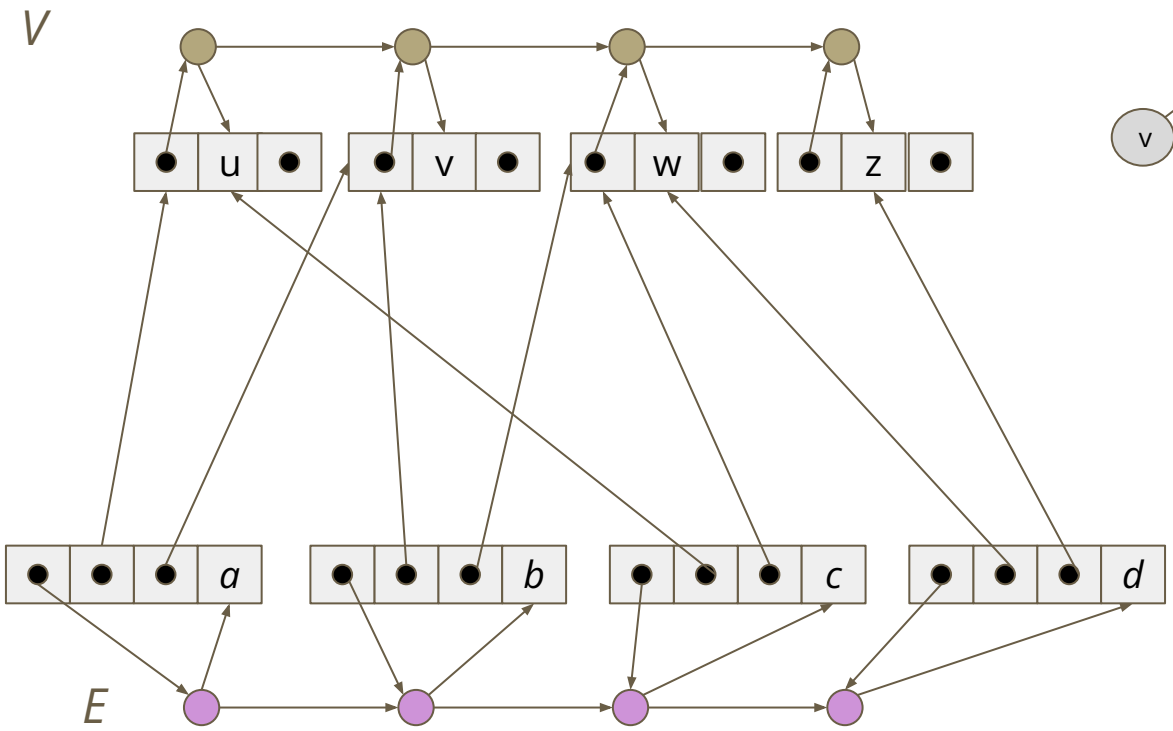


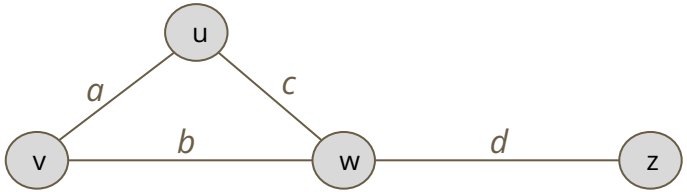
Estructuras para Grafos / Lista de Adyacencia



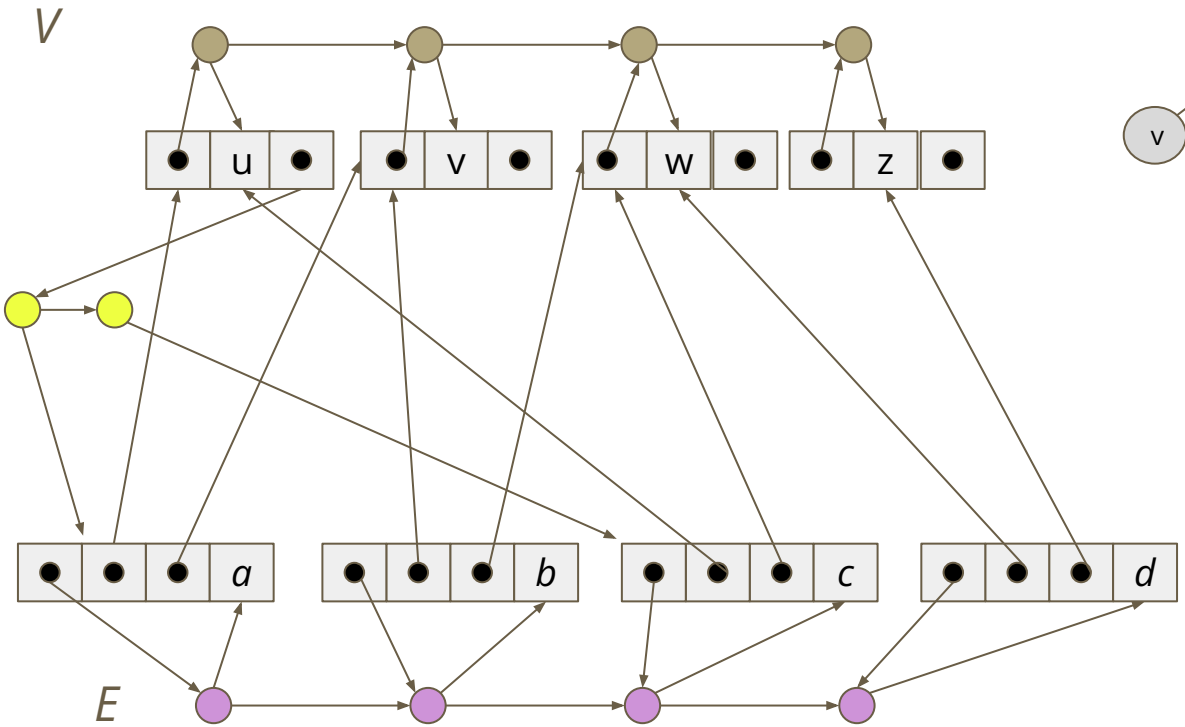


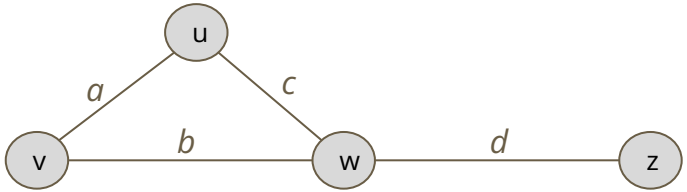
Estructuras para Grafos / Lista de Adyacencia



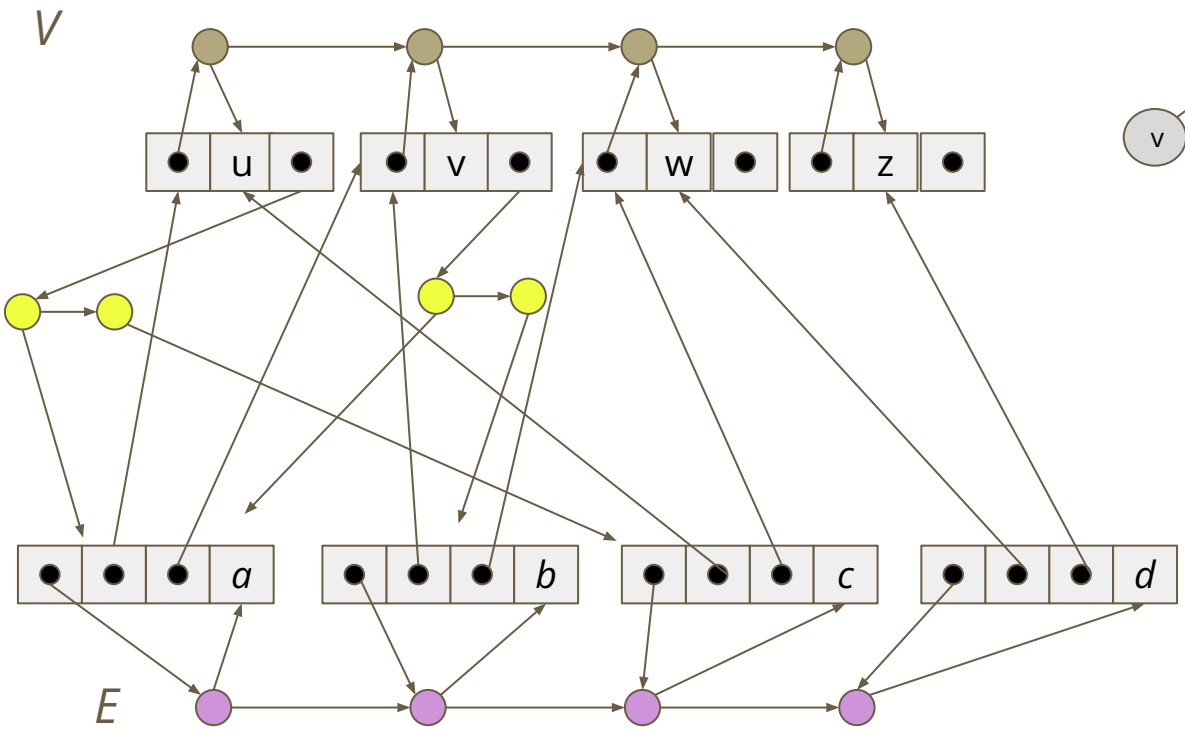


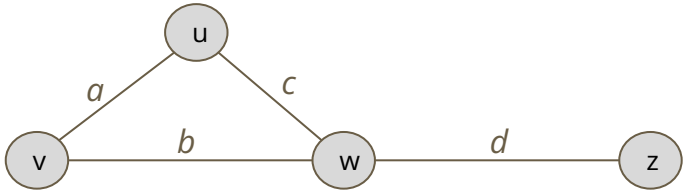
Estructuras para Grafos / Lista de Adyacencia



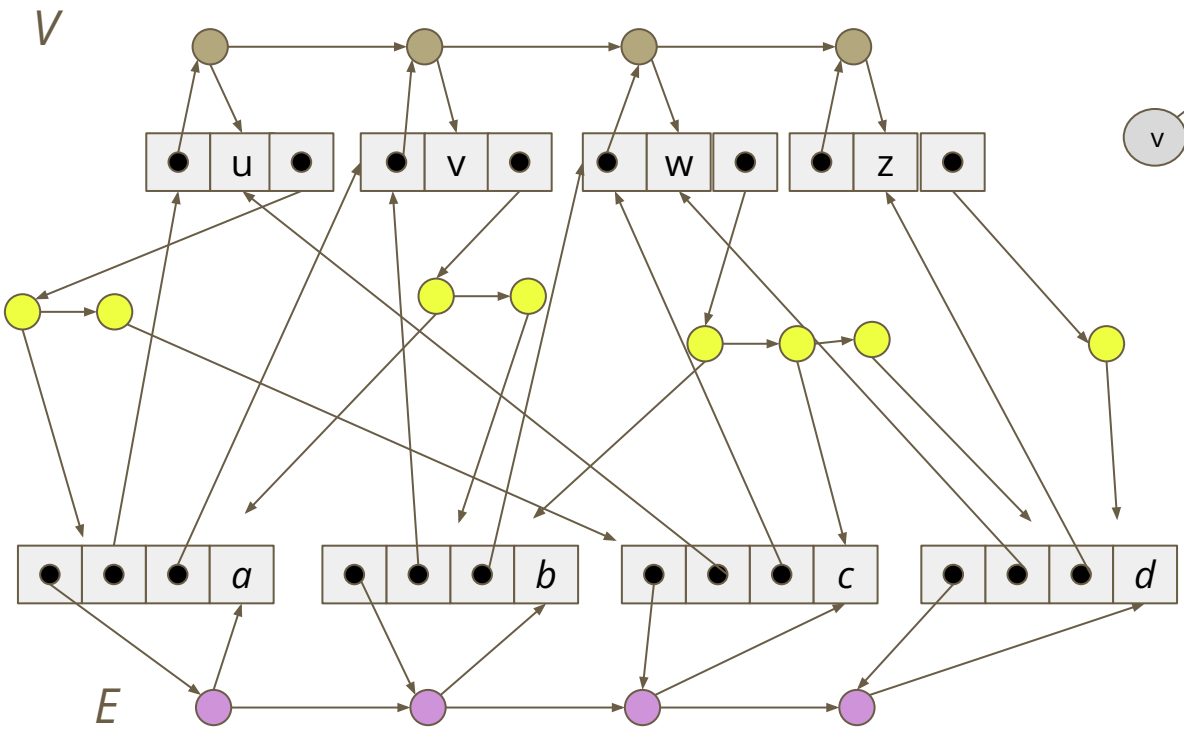


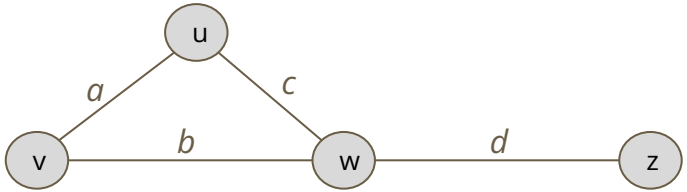
Estructuras para Grafos / Lista de Adyacencia





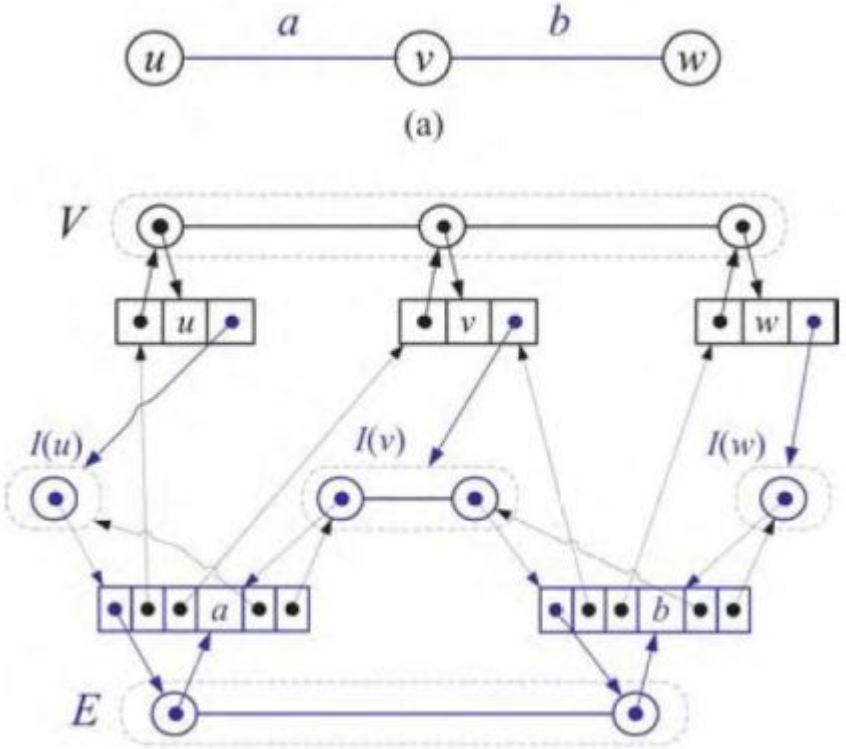
Estructuras para Grafos / Lista de Adyacencia



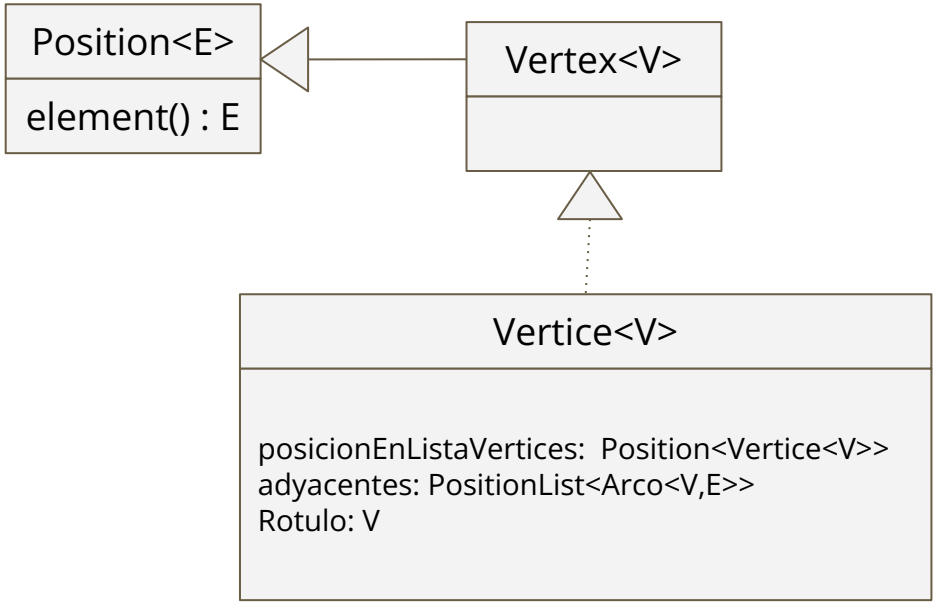


Estructuras para Grafos / Lista de Adyacencia

Otra variante



Estructuras para Grafos / Lista de Adyacencia



Vertice<V,E>

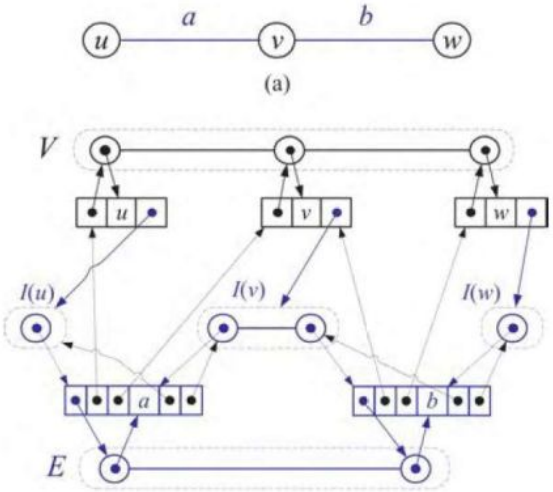
```
// Archivo: Vertice.java (o también se puede hacer clase anidada privada de la clase Grafo.java que implementa Graph)
public class Vertice<V,E> implements Vertex<V> {
        private V rotulo;
        private PositionList<Arco<V,E>> adyacentes;
        private Position<Vertice<V,E>> posicionEnNodos;
        public V element() { return rotulo; }
        public Vertice( V rotulo ) {
                this.rotulo = rotulo;
                adyacentes = new Lista<Arco<V,E>>();
        }
        // Setters y getters
        public void setRotulo(V nuevoRotulo) { ... }
        public PositionList<Arco<V,E>> getAdyacentes() { ... }
        public void
        setPosicionEnNodos(Position<Vertice<V,E>> p ) { ... }
        public Position<Vertice<V,E>> getPosicionEnNodos()
        { ... }
}
```

Arco<V,E>

```
public class Arco<V,E> implements Edge<E> {
        private E rotulo;
        private Vertice<V,E> v1, v2;
        private Position<Arco<V,E>> posicionEnArcos;
        private Position<Arco<V,E>> posicionEnIv1, posicionEnIv2;
        public Arco( E rotulo, Vertice<V,E> v1, Vertice<V,E> v2 )
        { /* solo setea atributos */ }
        public E element() { return rotulo; }
        // Setters y getters
        ...
}
```

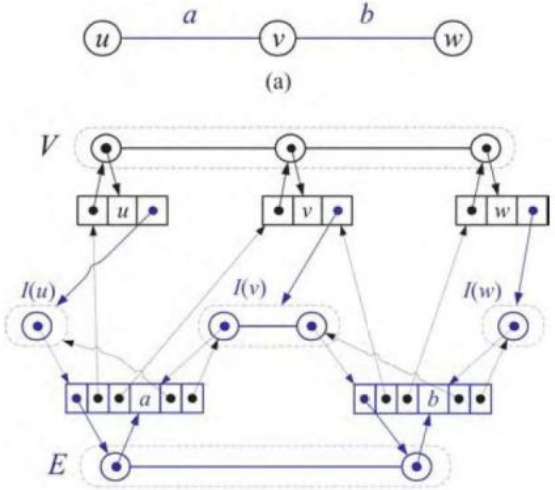
Grafo.java

```
public class Grafo<V,E> implements Graph<V,E>
{
	protected PositionList<Vertice<V,E>> nodos;
	protected PositionList<Arco<V,E>> arcos;
	public Grafo() {
		nodos = new Lista<Vertice<V,E>>();
		arcos = new Lista<Arco<V,E>>();
	}
	public Iterable<Vertex<V>> vertices() {
		PositionList<Vertex<V>> lista = new Lista<Vertex<V>>();
		for( Vertex<V> v : nodos )
			lista.addLast(v);
		return lista;
}
```



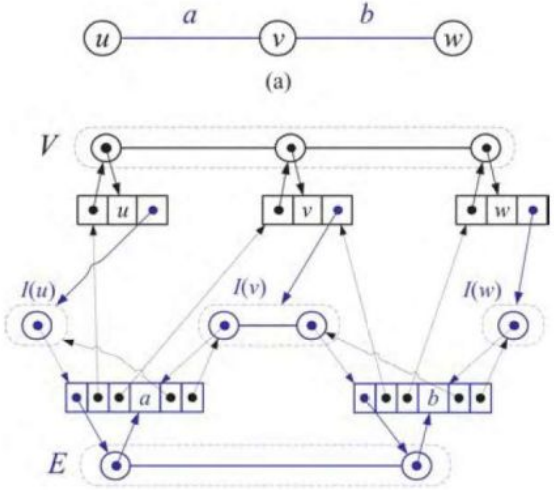
Grafo.java

```
public Iterable<Edge<E>> incidentEdges( Vertex<V> v) {
        PositionList<Edge<E>> lista = new Lista<Edge<E>>();
        Vertice<V,E> vert = (Vertice<V,E>) v;
        for( Edge<E> e : vert.getAdyacentes() )
                lista.addLast(e);
        return lista;
}
```

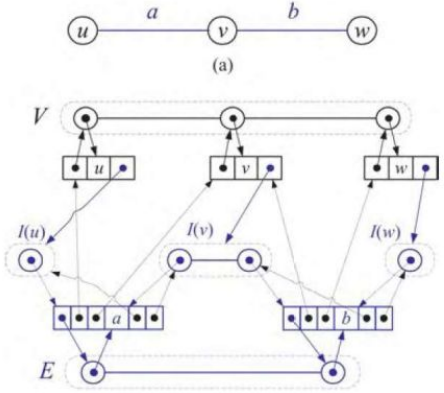


Grafo.java

```
//se crea un vértice con rótulo x y se insertar en la lista de vértices del grafo
//con una lista de adyacentes vacía
public Vertex<V> insertVertex(V x) {
        Vertice<V,E> v = new Vertice<V,E>(x);
        nodos.addLast(v);
        v.setPosicionEnNodos(nodos.last());
        return v;
}
```



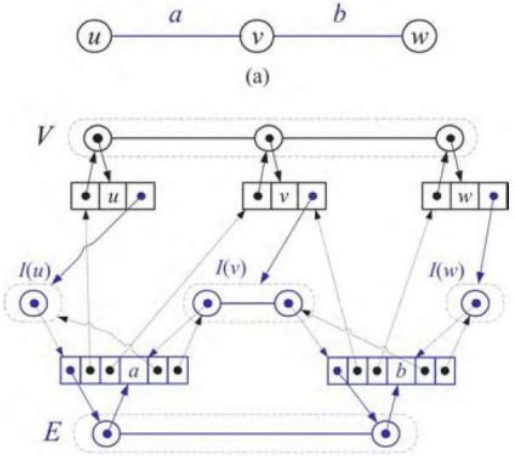
Grafo.java




```
public Edge<E> insertEdge(Vertex<V> v, Vertex<V> w, E x) {
        // Obtengo los vertices v y w:
        Vertice<V,E> vv = (Vertice<V,E>) v; Vertice<V,E> ww = (Vertice<V,E>) w;
        Arco<V,E> arco = new Arco<V,E>( x, vv, ww ); // Construyo un arco
        // Agrego el arco al final de la lista de adyacentes de v y anoto su posición:
        vv.getAdyacentes().addLast( arco );
        arco.setPosicionEnIv1( vv.getAdyacentes().last() );
        // Agrego el arco al final de la lista de adyacentes de w y anoto su posición:
        ww.getAdyacentes().addLast( arco );
        arco.setPosicionEnIv2( ww.getAdyacentes().last() );
        // Agrego el arco al final de lista de arcos y le seteo su posición:
        arcos.addLast(arco);
        arco.setPosEnArcos(arcos.last())
        return arco; // Retorno el arco creado
}
```

Grafo.java

```
public E removeEdge(Edge<E> e) {
        Arco<V,E> ee = (Arco<V,E>) e; // Recupero extremos del arco
        Vertice<V,E> v1 = ee.getV1(); Vertice<V,E> v2 = ee.getV2();
        // Elimino a e de la lista de adyacentes de v1:
        v1.getAdyacentes().remove(ee.getPosicionEnIv1());
        // Elimino a e de la lista de adyacentes de v2:
        v2.getAdyacentes().remove(ee.getPosicionEnIv2());
        // Elimino a e de la lista de arcos y retorno el rótulo del arco:
        Position<Arco<V,E>> pee = ee.getPosicionEnArcos();
        return arcos.remove(pee).element();
}
```



Estructuras para Grafos / Lista de Adyacencia

incidentEdges(Vertex<V> v);	?
opposite(Vertex<V> v, Edge<E> e);	O(1)
endVertices(Edge<E> e);	O(1)
areAdjacent(Vertex<V> v, Vertex<V> w);	?
replace(Vertex<V> v, V x);	O(1)
replace(Edge<E> e, E x);	O(1)
insertVertex(V x);	O(1)
insertEdge(Vertex<V> v, Vertex<V> w, E x);	O(1)
removeVertex(Vertex<V> v);	?
removeEdge(Edge<E> e); }	?

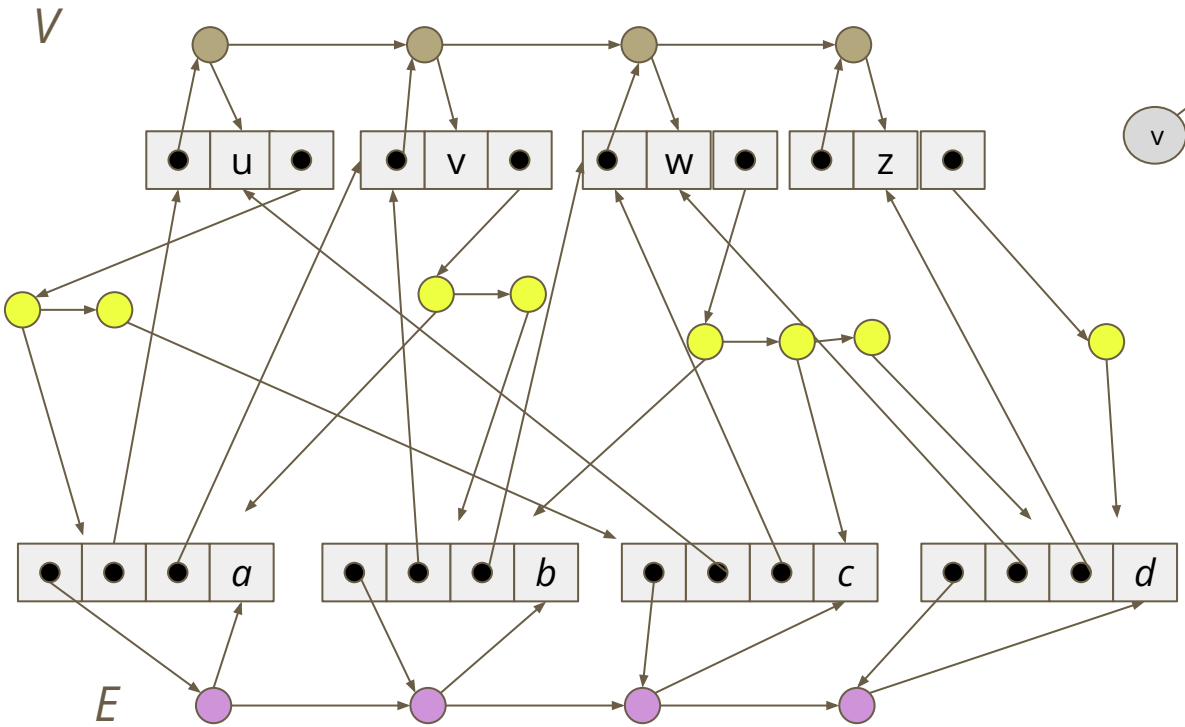
Dado un grafo G=(V,A). Sea n = #V y m = #A

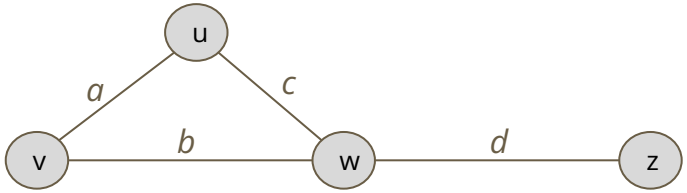
Estructuras para Grafos / Lista de Adyacencia

incidentEdges(Vertex<V> v);	O(grado(v))
opposite(Vertex<V> v, Edge<E> e);	O(1)
endVertices(Edge<E> e);	O(1)
areAdjacent(Vertex<V> v, Vertex<V> w);	O(grado(v)) → O(min(grado(v), min(grado(w))
replace(Vertex<V> v, V x);	O(1)
replace(Edge<E> e, E x);	O(1)
insertVertex(V x);	O(1)
insertEdge(Vertex<V> v, Vertex<V> w, E x);	O(1)
removeVertex(Vertex<V> v);	O(grado(v))
removeEdge(Edge<E> e); }	O(grado(v)) → O(1) con la segunda mejora

Dado un grafo G=(V,A). Sea n = #V y m = #A

Estructuras para Grafos / Lista de Adyacencia





¿Qué pasa si es un grafo dirigido?

Digrafo

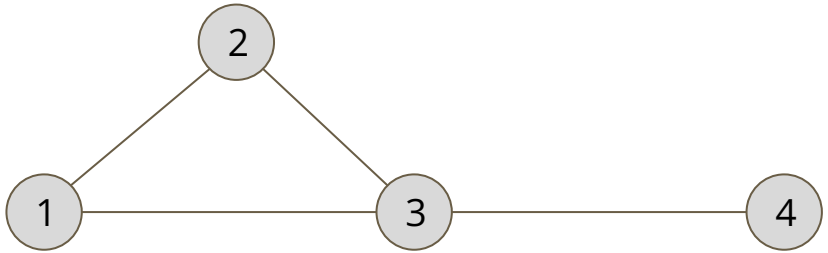
- Los arcos tienen dirección entonces (v,w) es diferente a (w,v)
- La operación insertEdge(v,w,x) inserta un arco dirigido de v a w con peso x
- Agregamos una nueva operación emergentEdges(v) que retorna un iterable con los arcos que emergen de v
- El almacenamiento de los arcos que inciden en un vértice puede o no mantenerse dependiendo de si nos interesa soportar la operación incidentEdges(v)

Estructuras de Datos para Grafos

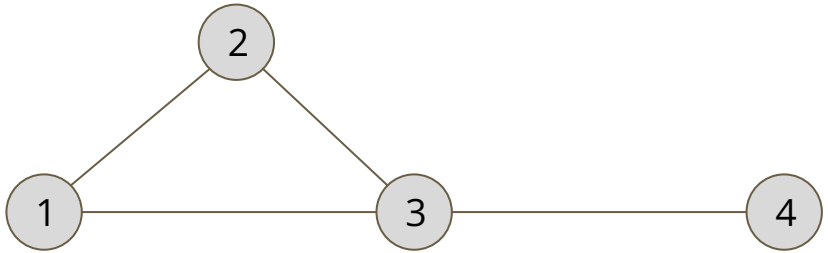
- Matriz de Adyacencia

  El grafo es una lista de vértices, una lista de arcos y una matriz donde cada componente (i,j) almacena un arco.

Estructuras para Grafos / Matriz de Adyacencia

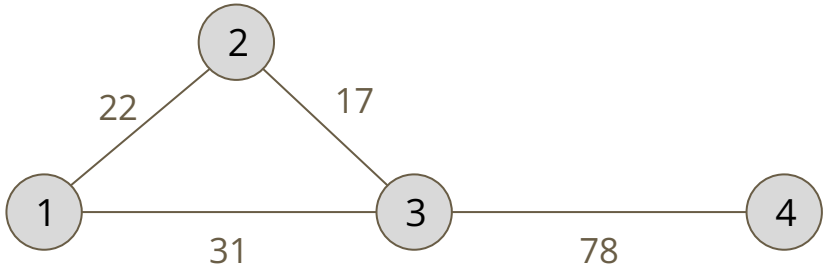


Estructuras para Grafos / Matriz de Adyacencia



	1	2	3	4
1	0	1	1	0
2	1	0	1	0
3	1	1	0	1
4	0	0	1	0

Estructuras para Grafos / Matriz de Adyacencia



	1	2	3	4
1	0	22	31	0
2	22	0	17	0
3	31	17	0	78
4	0	0	78	0

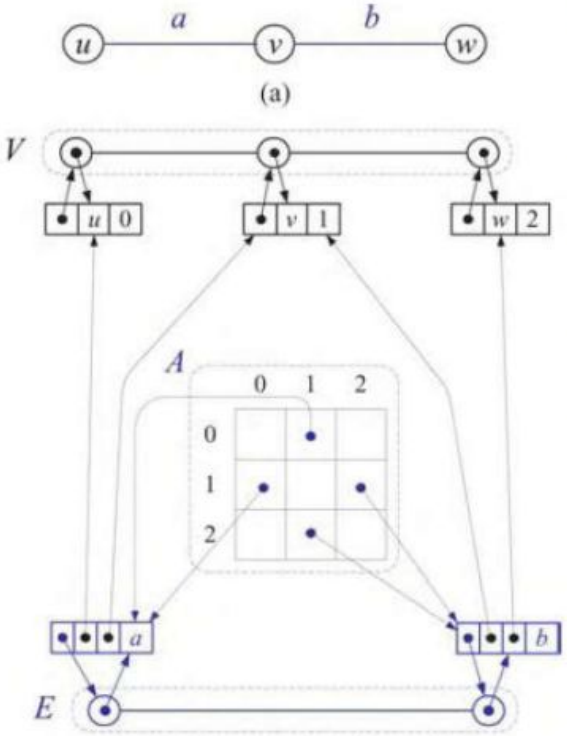
Estructuras para Grafos / Matriz de Adyacencia

En la representación de matriz de adyacencias además de la matriz de arcos tendremos una lista de vértices y una lista de arcos.

Cada vértice conoce su posición en la lista de vértices, el rótulo del vértice y su índice en la matriz de arcos.

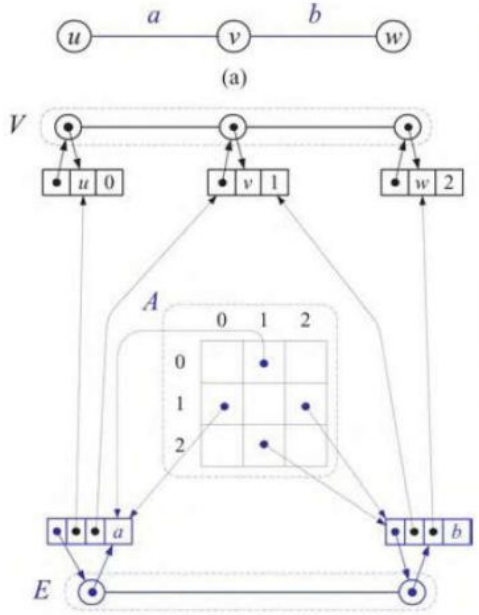
Como usamos Java, si tenemos n vértices, los índices de vértices van de 0 a n-1. Cada arco conoce su posición en la lista de arcos, los vértices que conecta y el peso (rótulo) del arco.

Estructuras para Grafos / Matriz de Adyacencia



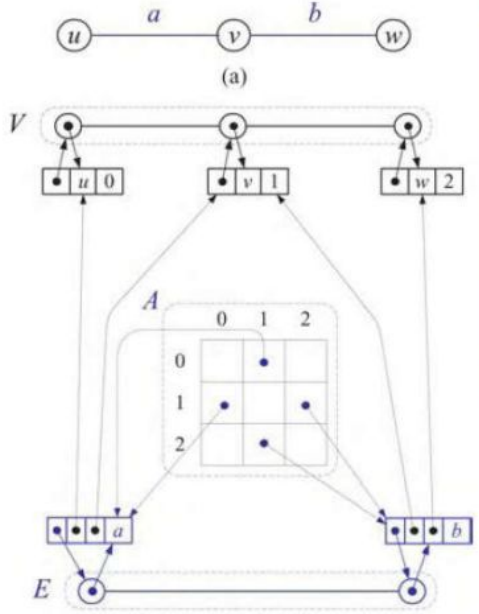
Implementación de grafo no dirigido con matriz de adyacencias

```
public class GrafoConMatriz<V,E> implements Graph<V,E> {
    protected PositionList<Vertex<V>> vertices;
    protected PositionList<Edge<E>> arcos;
    protected Edge<E> [][] matriz;
    protected int cantidadVertices;
```



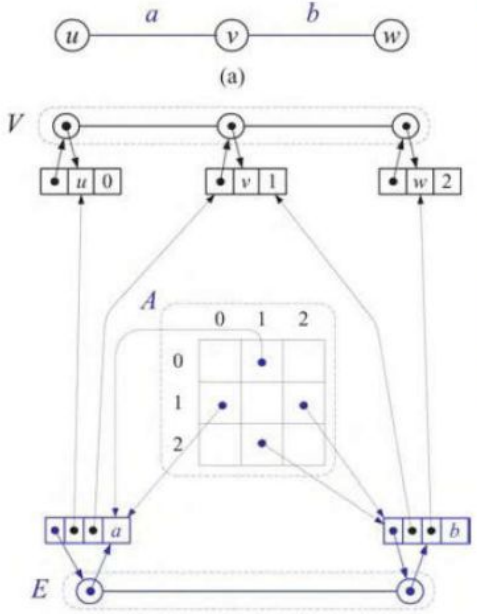
Implementación de grafo no dirigido con matriz de adyacencias

```
private class Vertice<V> implements Vertex<V> {
        private Position<Vertex<V>> posicionEnVertices;
        private V rotulo;
        private int indice;
        public Vertice( V rotulo, int indice ) {
                ... setear campos rotulo e índice... }
        // setters y getters:
        public void setPosicionEnVertices( Position<Vertex<V>> p ) { ... }
        public void setRotulo(V nuevoRotulo) { ... }
        public int getIndice() { ... }
        public Position<Vertex<V>> getPositionEnVertices() { ... }
        public V element() { ... }
}
```



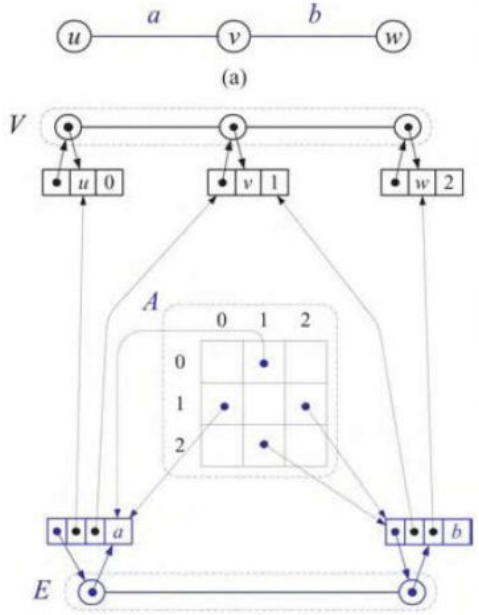
Implementación de grafo no dirigido con matriz de adyacencias

```
private class Arco<V,E> implements Edge<E> {
        private Position<Edge<E>> posicionEnArcos;
        private Vertice<V> v1, v2;
        private E rotulo;
        public Arco( E rotulo, Vertice<V> v1, Vertice<V> v2 ) {
                ...setear campos del arco... }
        // setters y getters:
        public void setPosicionEnArcos( Position<Edge<E>> p ) { ... }
        public E element() { ... }
        public Position<Edge<E>> getPosicionEnArcos() { ... }
        public Vertice<V> getV1() { ... }
        public Vertice<V> getV2() { ... }
        public void setRotulo(E nuevoRotulo) { ... }
}
```



Implementación de grafo no dirigido con matriz de adyacencias

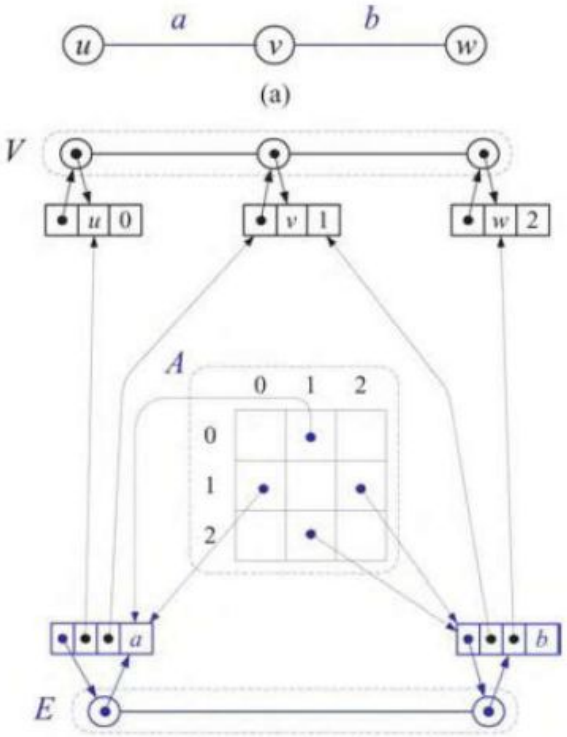
```
public Edge<E> insertEdge(Vertex<V> v, Vertex<V> w, E x) {
        // Cargo arco en la matriz:
        Vertice<V> vv = (Vertice<V>) v;
        Vertice<V> ww = (Vertice<V>) w;
        int fila = vv.getIndice();
        int col = ww.getIndice();
        Arco<V,E> arco = new Arco<V,E>( x, vv, ww );
        // Grafo no dirigido => matriz simétrica
        matriz[fila][col] = matriz[col][fila] = arco;
        // Cargo arco en la lista de arcos:
        arcos.addLast( arco );
        arco.setPosicionEnArcos( arcos.last() );
        return arco;
}
```



Estructuras para Grafos / Matriz de Adyacencia

incidentEdges(Vertex<V> v);
opposite(Vertex<V> v, Edge<E> e);
endVertices(Edge<E> e);
areAdjacent(Vertex<V> v, Vertex<V> w);
replace(Vertex<V> v, V x);
replace(Edge<E> e, E x);
insertVertex(V x);
insertEdge(Vertex<V> v, Vertex<V> w, E x);
removeVertex(Vertex<V> v);
removeEdge(Edge<E> e); }

Dado un grafo G=(V,A). Sea n = #V y m = #A



Tarea

Analice el tiempo de ejecución de las operaciones del grafo con la implementación de matriz de adyacencia.

Multigrafos

Un multigrafo es un grafo donde puede haber más de un arco conectando un par de vértices

- Con lista de arcos:
  - Lo soporta naturalmente
- Con lista de adyacencias:
  - Lo soporta naturalmente
- Con matriz de adyacencias:
  - La componente (i,j) de la matriz debe ser una lista de arcos

Bibliografía

Capítulo 13 de M. Goodrich & R. Tamassia, Data Structures and Algorithms in Java. Fourth Edition, John Wiley & Sons, 2006.

FIN.